

2. Operads, PROPs, and Symmetric Monoidal Foundations

Foundational role. This section introduces the algebraic languages that organize compositional digital structure before one specializes to Boolean closure, circuit synthesis, or symmetry analysis. Operads capture substitutional composition of operations with multiple inputs and one output; PROPs extend this perspective to operations with many inputs and many outputs; and colored operads refine both by tracking port types. Together, these form a symmetric monoidal foundation for the later chapters on closure, modularity, and equivalence.

Operads: substitution, units, symmetric actions, and algebras. An operad packages families of operations $\mathcal{O}(n)$ for $n \geq 0$, together with a unit, symmetric group actions, and a substitution law. Informally, an element of $\mathcal{O}(n)$ is an n -ary operation, and operadic composition expresses the replacement of each input of an operation by another operation of matching arity. The unit represents the identity operation, while the symmetric action records invariance under permutation of inputs.

This structure is well suited to syntax: trees of operations can be composed by grafting, and the operad axioms ensure that substitution is associative and unital up to the specified symmetric bookkeeping. In applications, an *algebra* over an operad interprets abstract operations as concrete functions on a carrier object, thereby turning syntax into semantics. This passage from formal expressions to realized behavior is the first recurring theme of the book.

From syntax to semantics: free structures and quotienting. Operads support a standard two-step workflow. First, one forms a free operadic structure generated by a chosen set of primitive operations; this captures all formal composites built from the generators. Second, one imposes equations or identifications to quotient by observationally irrelevant distinctions. The result is a semantic object in which different syntactic presentations may denote the same behavior.

This free-then-quotient pattern will reappear throughout the book. It is the algebraic analogue of building a circuit language from primitive gates and then identifying circuits that compute the same function, or building a proof language and then quotienting by proof equivalence. The point is not merely to generate expressions, but to organize the space of expressions by the relations that matter for composition and interpretation.

PROPs: strict symmetric monoidal categories on $\mathbb{N}$. Where operads model one-output operations, PROPs model general wiring patterns with multiple inputs and multiple outputs. A PROP may be viewed as a strict symmetric monoidal category whose objects are natural numbers $0, 1, 2, \dots$, with monoidal product given by addition. Morphisms $m \rightarrow n$ represent processes or circuits with m input ports and n output ports.

This viewpoint is especially natural for digital systems. A circuit is not just a function; it is a wiring diagram with explicit interface arity, internal composition, and tensoring of independent components. Sequential composition corresponds to plugging outputs into inputs, while the monoidal product corresponds to placing components side by side. Symmetry morphisms encode wire permutations, so that the formalism can express reordering of ports without changing the underlying process.

In this book, PROPs provide the ambient language for circuits as morphisms and for wiring diagrams as compositional artifacts. They are the right setting for discussing equivalence of circuit presentations, structural rewrites, and the interaction between algebraic laws and graphical syntax.

Colored operads and port typing. Colored operads generalize ordinary operads by allowing operations to have typed inputs and outputs. Each color represents a port type, and an operation is only admissible when its input and output colors match the intended interface. This refinement is essential whenever a system has heterogeneous components, such as mixed signal types, data/control separation, or typed resource flows.

Colored operads make explicit what is often implicit in informal circuit diagrams: not every wire can connect to every port. The typing discipline constrains substitution and ensures that composition respects interface compatibility. In this sense, colored operads are a natural bridge between abstract operadic syntax and typed engineering practice.

Relation to Lawvere theories. Lawvere theories provide another algebraic presentation of finitary algebraic structure, emphasizing finite products and operations with multiple inputs but a single output. They are closely related to operads and can often be compared through the lens of arity, substitution, and equational presentation. For single-sorted algebraic theories, Lawvere theories and operadic formalisms frequently encode similar information, though with different categorical emphases.

For the purposes of this book, the key point is that these frameworks are complementary descriptions of compositional structure. Operads emphasize substitutional syntax; PROPs emphasize general wiring and symmetric monoidal composition; colored operads emphasize typed interfaces; and Lawvere theories emphasize algebraic operations governed by finite-product structure. The choice of formalism depends on whether one wants to foreground syntax, wiring, typing, or equational semantics.

Observational equivalence on PROP morphisms. To compare PROP morphisms in a way that is sensitive to intended semantics, we introduce an observational equivalence relation $\varepsilon(r)$, indexed by a resolution or regime parameter r . Two morphisms are $\varepsilon(r)$ -equivalent when they are indistinguishable at the chosen observational scale. In the default logical regime, denoted $r = \text{spec}$, $\varepsilon(\text{spec})$ is taken to mean ordinary logical equivalence: two morphisms are identified when they induce the same specified behavior.

This notation is a reminder that equivalence is not always absolute; it may depend on the granularity at which a system is observed, tested, or specified. In later chapters, this idea will support multi-resolution reasoning about circuits, programs, and symmetries. For now, it serves as the semantic counterpart to the syntactic identifications imposed by quotienting.

Summary. Operads, PROPs, and colored operads provide the categorical infrastructure for the rest of the book. Operads organize substitution; PROPs organize multi-input, multi-output composition; colored operads enforce typed interfaces; and Lawvere theories situate these ideas within algebraic semantics. The recurring pattern is that of free generation followed by quotienting under an equivalence such as $\varepsilon(r)$, with $\varepsilon(\text{spec})$ serving as the default notion of logical sameness.